

开源软件基础

Beautiful Soup

Zhilei Ren



OSCAR Team, School of Software, Dalian University of Technology

November 23, 2017



Outline

- 1 快速开始
- 2 如何使用
- 3 对象的种类
- 4 遍历文档树
- 5 搜索文档树
- 6 修改文档树
- 7 输出
- 8 指定文档解析器
- 9 编码
- 10 解析部分文档

快速开始

下面的一段 HTML 代码将作为例子被多次用到. 这是爱丽丝梦游仙境的一段内容 (以后内容中简称为爱丽丝的文档):

快速开始

```
html_doc = """
<html><head><title>The Dormouse's story</title></head>
<body>
<p class="title"><b>The Dormouse's story</b></p>

<p class="story">Once upon a time there were three lit
<a href="http://example.com/elsie" class="sister" id="
<a href="http://example.com/lacie" class="sister" id="
<a href="http://example.com/tillie" class="sister" id=
and they lived at the bottom of a well.</p>

<p class="story">...</p>
"""
```

快速开始

使用 BeautifulSoup 解析这段代码, 能够得到一个 BeautifulSoup 的对象, 并能按照标准的缩进格式的结构输出:

快速开始

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html_doc)

print(soup.prettify())
# <html>
#   <head>
#     <title>
#       The Dormouse's story
#     </title>
#   </head>
#   <body>
#     <p class="title">
#       <b>
#         The Dormouse's story
```

快速开始

几个简单的浏览结构化数据的方法:

快速开始

```
soup.title  
# <title>The Dormouse's story</title>
```

```
soup.title.name  
# u'title'
```

```
soup.title.string  
# u'The Dormouse's story'
```

```
soup.title.parent.name  
# u'head'
```

```
soup.p  
# <p class="title"><b>The Dormouse's story</b></p>
```


快速开始

从文档中找到所有 `<a>` 标签的链接:

快速开始

```
for link in soup.find_all('a'):  
    print(link.get('href'))  
# http://example.com/elsie  
# http://example.com/lacie  
# http://example.com/tillie
```

快速开始

从文档中获取所有文字内容:

快速开始

```
print(soup.get_text())
# The Dormouse's story
#
# The Dormouse's story
#
# Once upon a time there were three little sisters; and
# Elsie,
# Lacie and
# Tillie;
# and they lived at the bottom of a well.
#
# ...
```

快速开始

这是你想要的吗？别着急，还有更好用的

Outline

- 1 快速开始
- 2 如何使用
- 3 对象的种类
- 4 遍历文档树
- 5 搜索文档树
- 6 修改文档树
- 7 输出
- 8 指定文档解析器
- 9 编码
- 10 解析部分文档

如何使用

将一段文档传入 BeautifulSoup 的构造方法, 就能得到一个文档的对象, 可以传入一段字符串或一个文件句柄.

如何使用

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(open("index.html"))

soup = BeautifulSoup("<html>data</html>")
```


如何使用

首先, 文档被转换成 Unicode, 并且 HTML 的实例都被转换成 Unicode 编码

如何使用

```
BeautifulSoup("Sacré; bleu!")  
<html><head></head><body>Sacré bleu!</body></html>
```

如何使用

然后,Beautiful Soup 选择最合适的解析器来解析这段文档, 如果手动指定解析器那么 Beautiful Soup 会选择指定的解析器来解析文档.(参考解析成 XML).

Outline

- 1 快速开始
- 2 如何使用
- 3 对象的种类**
- 4 遍历文档树
- 5 搜索文档树
- 6 修改文档树
- 7 输出
- 8 指定文档解析器
- 9 编码
- 10 解析部分文档

对象的种类

Beautiful Soup 将复杂 HTML 文档转换成一个复杂的树形结构, 每个节点都是 Python 对象, 所有对象可以归纳为 4 种: Tag , NavigableString , BeautifulSoup , Comment .

对象的种类

Tag 对象与 XML 或 HTML 原生文档中的 tag 相同:

对象的种类

```
soup = BeautifulSoup('<b class="boldest">Extremely bol  
tag = soup.b  
type(tag)  
# <class 'bs4.element.Tag'>
```

对象的种类

Tag 有很多方法和属性, 在遍历文档树和搜索文档树中有详细解释. 现在介绍一下 tag 中最重要的属性: name 和 attributes

对象的种类

每个 tag 都有自己的名字, 通过.name 来获取:

对象的种类

```
tag.name  
# u'b'
```

对象的种类

如果改变了 tag 的 name, 那将影响所有通过当前 BeautifulSoup 对象生成的 HTML 文档:

对象的种类

```
tag.name = "blockquote"  
tag  
# <blockquote class="boldest">Extremely bold</blockquote>
```

对象的种类

一个 tag 可能有很多个属性. tag `<b class="boldest">` 有一个 “class” 的属性, 值为 “boldest”. tag 的属性的操作方法与字典相同:

对象的种类

```
tag['class']  
# u'boldest'
```

对象的种类

也可以直接”点”取属性, 比如: `.attrs` :

对象的种类

```
tag.attrs  
# {u'class': u'boldest'}
```


对象的种类

tag 的属性可以被添加, 删除或修改. 再说一次, tag 的属性操作方法与字典一样

对象的种类

```
tag['class'] = 'verybold'
tag['id'] = 1
tag
# <blockquote class="verybold" id="1">Extremely bold</

del tag['class']
del tag['id']
tag
# <blockquote>Extremely bold</blockquote>

tag['class']
# KeyError: 'class'
print(tag.get('class'))
# None
```

对象的种类

HTML 4 定义了一系列可以包含多个值的属性. 在 HTML5 中移除了一些, 却增加更多. 最常见的多值的属性是 class (一个 tag 可以有多个 CSS 的 class). 还有一些属性 rel , rev , accept-charset , headers , accesskey . 在 Beautiful Soup 中多值属性的返回类型是 list:

对象的种类

```
css_soup = BeautifulSoup('<p class="body strikeout"></p>')
css_soup.p['class']
# ["body", "strikeout"]
```

```
css_soup = BeautifulSoup('<p class="body"></p>')
css_soup.p['class']
# ["body"]
```

对象的种类

如果某个属性看起来好像有多个值,但在任何版本的 HTML 定义中都没有被定义为多值属性,那么 BeautifulSoup 会将这个属性作为字符串返回

对象的种类

```
id_soup = BeautifulSoup('<p id="my id"></p>')  
id_soup.p['id']  
# 'my id'
```

对象的种类

将 tag 转换成字符串时, 多值属性会合并为一个值

对象的种类

```
rel_soup = BeautifulSoup('<p>Back to the <a rel="index  
rel_soup.a['rel']  
# ['index']  
rel_soup.a['rel'] = ['index', 'contents']  
print(rel_soup.p)  
# <p>Back to the <a rel="index contents">homepage</a><
```


对象的种类

如果转换的文档是 XML 格式, 那么 tag 中不包含多值属性

对象的种类

```
xml_soup = BeautifulSoup('<p class="body strikeout"></p>')
xml_soup.p['class']
# u'body strikeout'
```

对象的种类

字符串常被包含在 tag 内.Beautiful Soup 用 NavigableString 类来包装 tag 中的字符串:

对象的种类

```
tag.string  
# u'Extremely bold'  
type(tag.string)  
# <class 'bs4.element.NavigableString'>
```

对象的种类

一个 `NavigableString` 字符串与 Python 中的 `Unicode` 字符串相同, 并且还支持包含在遍历文档树和搜索文档树中的一些特性. 通过 `unicode()` 方法可以直接将 `NavigableString` 对象转换成 `Unicode` 字符串:

对象的种类

```
unicode_string = unicode(tag.string)
unicode_string
# u'Extremely bold'
type(unicode_string)
# <type 'unicode'>
```

对象的种类

tag 中包含的字符串不能编辑, 但是可以被替换成其它的字符串, 用 `replace_with()` 方法:

对象的种类

```
tag.string.replace_with("No longer bold")
tag
# <blockquote>No longer bold</blockquote>
```


对象的种类

`NavigableString` 对象支持遍历文档树和搜索文档树中定义的大部分属性, 并非全部. 尤其是, 一个字符串不能包含其它内容 (tag 能够包含字符串或是其它 tag), 字符串不支持 `.contents` 或 `.string` 属性或 `find()` 方法.

对象的种类

如果想在 Beautiful Soup 之外使用 NavigableString 对象, 需要调用 `unicode()` 方法, 将该对象转换成普通的 Unicode 字符串, 否则就算 Beautiful Soup 已方法已经执行结束, 该对象的输出也会带有对象的引用地址. 这样会浪费内存.

对象的种类

BeautifulSoup 对象表示的是一个文档的全部内容. 大部分时候, 可以把它当作 Tag 对象, 它支持遍历文档树和搜索文档树中描述的大部分的方法.

对象的种类

因为 BeautifulSoup 对象并不是真正的 HTML 或 XML 的 tag, 所以它没有 name 和 attribute 属性. 但有时查看它的.name 属性是很方便的, 所以 BeautifulSoup 对象包含了一个值为 “[document]” 的特殊属性.name

对象的种类

```
soup.name  
# u'[document]'
```

对象的种类

Tag , NavigableString , BeautifulSoup 几乎覆盖了 html 和 xml 中的所有内容, 但是还有一些特殊对象. 容易让人担心的内容是文档的注释部分:

对象的种类

```
markup = "<b><!--Hey, buddy. Want to buy a used parser  
soup = BeautifulSoup(markup)  
comment = soup.b.string  
type(comment)  
# <class 'bs4.element.Comment'>
```

对象的种类

Comment 对象是一个特殊类型的 NavigableString 对象:

对象的种类

```
comment  
# u'Hey, buddy. Want to buy a used parser'
```

对象的种类

但是当它出现在 HTML 文档中时, Comment 对象会使用特殊的格式输出:

对象的种类

```
print(soup.b.prettify())  
# <b>  
# <!--Hey, buddy. Want to buy a used parser?-->  
# </b>
```

对象的种类

Beautiful Soup 中定义的其它类型都可能会出现在 XML 的文档中: CData , ProcessingInstruction , Declaration , Doctype . 与 Comment 对象类似, 这些类都是 NavigableString 的子类, 只是添加了一些额外的方法的字符串独享. 下面是用 CDATA 来替代注释的例子:

对象的种类

```
from bs4 import CData
cdata = CData("A CDATA block")
comment.replace_with(cdata)

print(soup.b.prettify())
# <b>
# <![CDATA[A CDATA block]]>
# </b>
```

Outline

- 1 快速开始
- 2 如何使用
- 3 对象的种类
- 4 遍历文档树**
- 5 搜索文档树
- 6 修改文档树
- 7 输出
- 8 指定文档解析器
- 9 编码
- 10 解析部分文档

遍历文档树

还拿”爱丽丝梦游仙境”的文档来做例子:

遍历文档树

```
html_doc = """
<html><head><title>The Dormouse's story</title></head>

<p class="title"><b>The Dormouse's story</b></p>

<p class="story">Once upon a time there were three lit
<a href="http://example.com/elsie" class="sister" id="
<a href="http://example.com/lacie" class="sister" id="
<a href="http://example.com/tillie" class="sister" id=
and they lived at the bottom of a well.</p>

<p class="story">...</p>
"""
```


遍历文档树

通过这段例子来演示怎样从文档的一段内容找到另一段内容

遍历文档树

一个 Tag 可能包含多个字符串或其它的 Tag, 这些都是这个 Tag 的子节点. Beautiful Soup 提供了许多操作和遍历子节点的属性.

遍历文档树

注意: Beautiful Soup 中字符串节点不支持这些属性, 因为字符串没有子节点

遍历文档树

操作文档树最简单的方法就是告诉它你想获取的 tag 的 name. 如果想获取 `<head>` 标签, 只要用 `soup.head` :

遍历文档树

```
soup.head
# <head><title>The Dormouse's story</title></head>

soup.title
# <title>The Dormouse's story</title>
```

遍历文档树

这是个获取 tag 的小窍门, 可以在文档树的 tag 中多次调用这个方法. 下面的代码可以获取 `<body>` 标签中的第一个 `<b>` 标签:

遍历文档树

```
soup.body.b  
# <b>The Dormouse's story</b>
```

遍历文档树

通过点取属性的方式只能获得当前名字的第一个 tag:

遍历文档树

```
soup.a
```

```
# <a class="sister" href="http://example.com/elsie" id
```

遍历文档树

如果想要得到所有的 `<a>` 标签, 或是通过名字得到比一个 tag 更多的内容的时候, 就需要用到 Searching the tree 中描述的方法, 比如: `find_all()`

遍历文档树

```
soup.find_all('a')  
# [<a class="sister" href="http://example.com/elsie" i  
#  <a class="sister" href="http://example.com/lacie" i  
#  <a class="sister" href="http://example.com/tillie"
```

遍历文档树

tag 的 .contents 属性可以将 tag 的子节点以列表的方式输出:

遍历文档树

```
head_tag = soup.head
head_tag
# <head><title>The Dormouse's story</title></head>

head_tag.contents
[<title>The Dormouse's story</title>]

title_tag = head_tag.contents[0]
title_tag
# <title>The Dormouse's story</title>
title_tag.contents
# [u'The Dormouse's story']
```

遍历文档树

BeautifulSoup 对象本身一定会包含子节点, 也就是说 `<html>` 标签也是 BeautifulSoup 对象的子节点:

遍历文档树

```
len(soup.contents)
# 1
soup.contents[0].name
# u'html'
```

遍历文档树

字符串没有.contents 属性, 因为字符串没有子节点:

遍历文档树

```
text = title_tag.contents[0]
text.contents
# AttributeError: 'NavigableString' object has no attr
```

遍历文档树

通过 tag 的.children 生成器, 可以对 tag 的子节点进行循环:

遍历文档树

```
for child in title_tag.children:  
    print(child)  
    # The Dormouse's story
```

遍历文档树

.contents 和.children 属性仅包含 tag 的直接子节点. 例如,<head> 标签只有一个直接子节点 <title>

遍历文档树

```
head_tag.contents  
# [<title>The Dormouse's story</title>]
```

遍历文档树

但是 `<title>` 标签也包含一个子节点: 字符串 “The Dormouse’ s story” , 这种情况下字符串 “The Dormouse’ s story” 也属于 `<head>` 标签的子孙节点. `.descendants` 属性可以对所有 tag 的子孙节点进行递归循环 [5]:

遍历文档树

```
for child in head_tag.descendants:  
    print(child)  
    # <title>The Dormouse's story</title>  
    # The Dormouse's story
```

遍历文档树

上面的例子中, `<head>` 标签只有一个子节点, 但是有 2 个子孙节点:`<head>` 节点和 `<head>` 的子节点, BeautifulSoup 有一个直接子节点 (`<html>` 节点), 却有很多子孙节点:

遍历文档树

```
len(list(soup.children))  
# 1  
len(list(soup.descendants))  
# 25
```

遍历文档树

如果 tag 只有一个 NavigableString 类型子节点, 那么这个 tag 可以使用.string 得到子节点:

遍历文档树

```
title_tag.string  
# u'The Dormouse's story'
```

遍历文档树

如果一个 tag 仅有一个子节点, 那么这个 tag 也可以使用.string 方法, 输出结果与当前唯一子节点的.string 结果相同:

遍历文档树

```
head_tag.contents  
# [<title>The Dormouse's story</title>]  
  
head_tag.string  
# u'The Dormouse's story'
```

遍历文档树

如果 tag 包含了多个子节点,tag 就无法确定.string 方法应该调用哪个子节点的内容, .string 的输出结果是 None :

遍历文档树

```
print(soup.html.string)  
# None
```

遍历文档树

如果 tag 中包含多个字符串 [2] , 可以使用.strings 来循环获取:

遍历文档树

```
for string in soup.strings:
    print(repr(string))
    # u"The Dormouse's story"
    # u'\n\n'
    # u"The Dormouse's story"
    # u'\n\n'
    # u'Once upon a time there were three little sisters'
    # u'Elsie'
    # u',\n'
    # u'Lacie'
    # u' and\n'
    # u'Tillie'
    # u';\nand they lived at the bottom of a well'
    # u'\n\n'
```

遍历文档树

输出的字符串中可能包含了很多空格或空行, 使用`.stripped_strings` 可以去除多余空白内容:

遍历文档树

```
for string in soup.stripped_strings:
    print(repr(string))
    # u"The Dormouse's story"
    # u"The Dormouse's story"
    # u'Once upon a time there were three little siste
    # u'Elsie'
    # u','
    # u'Lacie'
    # u'and'
    # u'Tillie'
    # u';\nand they lived at the bottom of a well.'
    # u'...'
```

遍历文档树

全部是空格的行会被忽略掉, 段首和段末的空白会被删除

遍历文档树

继续分析文档树, 每个 tag 或字符串都有父节点: 被包含在某个 tag 中

遍历文档树

通过.parent 属性来获取某个元素的父节点. 在例子“爱丽丝”的文档中,<head> 标签是 <title> 标签的父节点:

遍历文档树

```
title_tag = soup.title
title_tag
# <title>The Dormouse's story</title>
title_tag.parent
# <head><title>The Dormouse's story</title></head>
```

遍历文档树

文档 title 的字符串也有父节点:<title> 标签

遍历文档树

```
title_tag.string.parent  
# <title>The Dormouse's story</title>
```

遍历文档树

文档的顶层节点比如 `<html>` 的父节点是 BeautifulSoup 对象:

遍历文档树

```
html_tag = soup.html  
type(html_tag.parent)  
# <class 'bs4.BeautifulSoup'>
```

遍历文档树

BeautifulSoup 对象的.parent 是 None:

遍历文档树

```
print(soup.parent)
# None
```

遍历文档树

通过元素的.parents 属性可以递归得到元素的所有父辈节点, 下面的例子使用了.parents 方法遍历了 <a> 标签到根节点的所有节点.

遍历文档树

```
link = soup.a
link
# <a class="sister" href="http://example.com/elsie" id
for parent in link.parents:
    if parent is None:
        print(parent)
    else:
        print(parent.name)

# p
# body
# html
# [document]
# None
```

遍历文档树

看一段简单的例子:

遍历文档树

```
sibling_soup = BeautifulSoup("<a><b>text1</b><c>text2</c></a></body></html>")
print(sibling_soup.prettify())

# <html>
#   <body>
#     <a>
#       <b>
#         text1
#       </b>
#     <c>
#       text2
#     </c>
#   </a>
# </body>
# </html>
```

遍历文档树

因为 `<b>` 标签和 `<c>` 标签是同一层: 他们是同一个元素的子节点, 所以 `<b>` 和 `<c>` 可以被称为兄弟节点. 一段文档以标准格式输出时, 兄弟节点有相同的缩进级别. 在代码中也可以使用这种关系.

遍历文档树

在文档树中, 使用`.next_sibling` 和`.previous_sibling` 属性来查询兄弟节点:

遍历文档树

```
sibling_soup.b.next_sibling
```

```
# <c>text2</c>
```

```
sibling_soup.c.previous_sibling
```

```
# <b>text1</b>
```

遍历文档树

 标签有.next_sibling 属性, 但是没有.previous_sibling 属性, 因为 标签在同级节点中是第一个. 同理,<c> 标签有.previous_sibling 属性, 却没有.next_sibling 属性:

遍历文档树

```
print(sibling_soup.b.previous_sibling)
# None
print(sibling_soup.c.next_sibling)
# None
```

遍历文档树

例子中的字符串“text1”和“text2”不是兄弟节点, 因为它们的父节点不同:

遍历文档树

```
sibling_soup.b.string  
# u'text1'  
  
print(sibling_soup.b.string.next_sibling)  
# None
```


遍历文档树

实际文档中的 tag 的 `.next_sibling` 和 `.previous_sibling` 属性通常是字符串或空白. 看看“爱丽丝”文档:

遍历文档树

```
<a href="http://example.com/elsie" class="sister" id="1"
<a href="http://example.com/lacie" class="sister" id="2"
<a href="http://example.com/tillie" class="sister" id="3"
```

遍历文档树

如果以为第一个 `<a>` 标签的 `.next_sibling` 结果是第二个 `<a>` 标签, 那就错了, 真实结果是第一个 `<a>` 标签和第二个 `<a>` 标签之间的顿号和换行符:

遍历文档树

```
link = soup.a
link
# <a class="sister" href="http://example.com/elsie" id=

link.next_sibling
# u', \n'
```

遍历文档树

第二个 `<a>` 标签是顿号的`.next_sibling` 属性:

遍历文档树

```
link.next_sibling.next_sibling  
# <a class="sister" href="http://example.com/lacie" id
```

遍历文档树

通过`.next_siblings` 和`.previous_siblings` 属性可以对当前节点的兄弟节点迭代输出:

遍历文档树

```
for sibling in soup.a.next_siblings:
    print(repr(sibling))
    # u',\n'
    # <a class="sister" href="http://example.com/lacie"
    # u' and\n'
    # <a class="sister" href="http://example.com/tilli
    # u'; and they lived at the bottom of a well.'
    # None

for sibling in soup.find(id="link3").previous_siblings:
    print(repr(sibling))
    # ' and\n'
    # <a class="sister" href="http://example.com/lacie
    # u',\n'
```


遍历文档树

看一下“爱丽丝”文档:

遍历文档树

```
<html><head><title>The Dormouse's story</title></head>  
<p class="title"><b>The Dormouse's story</b></p>
```

遍历文档树

HTML 解析器把这段字符串转换成一连串的事件：“打开 <html> 标签”，“打开一个 <head> 标签”，“打开一个 <title> 标签”，“添加一段字符串”，“关闭 <title> 标签”，“打开 <p> 标签”，等等.Beautiful Soup 提供了重现解析器初始化过程的方法。

遍历文档树

.next_element 属性指向解析过程中下一个被解析的对象 (字符串或 tag), 结果可能与.next_sibling 相同, 但通常是不一样的.

遍历文档树

这是“爱丽丝”文档中最后一个 `<a>` 标签, 它的 `.next_sibling` 结果是一个字符串, 因为当前的解析过程 [2] 因为当前的解析过程因为遇到了 `<a>` 标签而中断了:

遍历文档树

```
last_a_tag = soup.find("a", id="link3")
last_a_tag
# <a class="sister" href="http://example.com/tillie" i
last_a_tag.next_sibling
# ';' and they lived at the bottom of a well.'
```

遍历文档树

但这个 `<a>` 标签的 `.next_element` 属性结果是在 `<a>` 标签被解析之后的解析内容, 不是 `<a>` 标签后的句子部分, 应该是字符串 "Tillie" :

遍历文档树

```
last_a_tag.next_element  
# u'Tillie'
```


遍历文档树

这是因为在原始文档中, 字符串 “Tillie” 在分号前出现, 解析器先进入 `<a>` 标签, 然后是字符串 “Tillie”, 然后关闭 `</a>` 标签, 然后是分号和剩余部分. 分号与 `<a>` 标签在同一层级, 但是字符串 “Tillie” 会被先解析.

遍历文档树

.previous_element 属性刚好与.next_element 相反, 它指向当前被解析的对象的`前一个`解析对象:

遍历文档树

```
last_a_tag.previous_element  
# u' and\n'  
last_a_tag.previous_element.next_element  
# <a class="sister" href="http://example.com/tillie" i
```

遍历文档树

通过`.next_elements` 和`.previous_elements` 的迭代器就可以向前或向后访问文档的解析内容, 就好像文档正在被解析一样:

遍历文档树

```
for element in last_a_tag.next_elements:
    print(repr(element))
# u'Tillie'
# u';\nand they lived at the bottom of a well.'
# u'\n\n'
# <p class="story">...</p>
# u'... '
# u'\n'
# None
```

Outline

- 1 快速开始
- 2 如何使用
- 3 对象的种类
- 4 遍历文档树
- 5 搜索文档树**
- 6 修改文档树
- 7 输出
- 8 指定文档解析器
- 9 编码
- 10 解析部分文档

搜索文档树

Beautiful Soup 定义了很多搜索方法, 这里着重介绍 2 个: `find()` 和 `find_all()`. 其它方法的参数和用法类似, 请读者举一反三.

搜索文档树

再以“爱丽丝”文档作为例子：

搜索文档树

```
html_doc = """
<html><head><title>The Dormouse's story</title></head>

<p class="title"><b>The Dormouse's story</b></p>

<p class="story">Once upon a time there were three lit
<a href="http://example.com/elsie" class="sister" id="
<a href="http://example.com/lacie" class="sister" id="
<a href="http://example.com/tillie" class="sister" id=
and they lived at the bottom of a well.</p>

<p class="story">...</p>
"""
```

搜索文档树

使用 `find_all()` 类似的方法可以查找到想要查找的文档内容

搜索文档树

介绍 `find_all()` 方法前, 先介绍一下过滤器的类型 [3], 这些过滤器贯穿整个搜索的 API. 过滤器可以被用在 tag 的 name 中, 节点的属性中, 字符串中或他们的混合中.

搜索文档树

最简单的过滤器是字符串. 在搜索方法中传入一个字符串参数,Beautiful Soup 会查找与字符串完整匹配的内容, 下面的例子用于查找文档中所有的 标签:

搜索文档树

```
soup.find_all('b')  
# [<b>The Dormouse's story</b>]
```

搜索文档树

如果传入字节码参数,Beautiful Soup 会当作 UTF-8 编码, 可以传入一段 Unicode 编码来避免 BeautifulSoup 解析编码出错

搜索文档树

如果传入正则表达式作为参数,Beautiful Soup 会通过正则表达式的 `match()` 来匹配内容. 下面例子中找出所有以 `b` 开头的标签, 这表示 `<body>` 和 `<b>` 标签都应该被找到:

搜索文档树

```
import re
for tag in soup.find_all(re.compile("^b")):
    print(tag.name)
# body
# b
```


搜索文档树

下面代码找出所有名字中包含”t”的标签:

搜索文档树

```
for tag in soup.find_all(re.compile("t")):  
    print(tag.name)  
# html  
# title
```

搜索文档树

如果传入列表参数,Beautiful Soup 会将与列表中任一元素匹配的内容返回. 下面代码找到文档中所有 <a> 标签和 标签:

搜索文档树

```
soup.find_all(["a", "b"])  
# [<b>The Dormouse's story</b>,  
#  <a class="sister" href="http://example.com/elsie" i  
#  <a class="sister" href="http://example.com/lacie" i  
#  <a class="sister" href="http://example.com/tillie"
```

搜索文档树

True 可以匹配任何值, 下面代码查找到所有的 tag, 但是不会返回字符串节点

搜索文档树

```
for tag in soup.find_all(True):  
    print(tag.name)  
  
# html  
# head  
# title  
# body  
# p  
# b  
# p  
# a  
# a  
# a  
# p
```

搜索文档树

如果没有合适过滤器, 那么还可以定义一个方法, 方法只接受一个元素参数 [4], 如果这个方法返回 True 表示当前元素匹配并且被找到, 如果不是则返回 False

搜索文档树

下面方法校验了当前元素, 如果包含 class 属性却不包含 id 属性, 那么将返回 True:

搜索文档树

```
def has_class_but_no_id(tag):  
    return tag.has_attr('class') and not tag.has_attr('id')
```

搜索文档树

将这个方法作为参数传入 `find_all()` 方法, 将得到所有 `<p>` 标签:

搜索文档树

```
soup.find_all(has_class_but_no_id)
# [<p class="title"><b>The Dormouse's story</b></p>,
#  <p class="story">Once upon a time there were...</p>
#  <p class="story">...</p>]
```

搜索文档树

返回结果中只有 `<p>` 标签没有 `<a>` 标签, 因为 `<a>` 标签还定义了” id”, 没有返回 `<html>` 和 `<head>`, 因为 `<html>` 和 `<head>` 中没有定义” class” 属性.

搜索文档树

下面代码找到所有被文字包含的节点内容:

搜索文档树

```
from bs4 import NavigableString
def surrounded_by_strings(tag):
    return (isinstance(tag.next_element, NavigableString)
            and isinstance(tag.previous_element, NavigableString))

for tag in soup.find_all(surrounded_by_strings):
    print tag.name

# p
# a
# a
# a
# p
```

搜索文档树

现在来了解一下搜索方法的细节

搜索文档树

```
find_all( name , attrs , recursive , text , **kwargs )
```


搜索文档树

`find_all()` 方法搜索当前 tag 的所有 tag 子节点, 并判断是否符合过滤器的条件. 这里有几个例子:

搜索文档树

```
soup.find_all("title")
# [<title>The Dormouse's story</title>]

soup.find_all("p", "title")
# [<p class="title"><b>The Dormouse's story</b></p>]

soup.find_all("a")
# [<a class="sister" href="http://example.com/elsie" i
#  <a class="sister" href="http://example.com/lacie" i
#  <a class="sister" href="http://example.com/tillie" i

soup.find_all(id="link2")
# [<a class="sister" href="http://example.com/lacie" i
```

搜索文档树

有几个方法很相似, 还有几个方法是新的, 参数中的 text 和 id 是什么含义? 为什么 `find_all("p", "title")` 返回的是 CSS Class 为 "title" 的 `<p>` 标签? 我们来仔细看一下 `find_all()` 的参数

搜索文档树

name 参数可以查找所有名字为 name 的 tag, 字符串对象会被自动忽略掉.

搜索文档树

简单的用法如下:

搜索文档树

```
soup.find_all("title")  
# [<title>The Dormouse's story</title>]
```

搜索文档树

重申: 搜索 name 参数的值可以使任一类型的过滤器, 字符窜, 正则表达式, 列表, 方法或是 True .

搜索文档树

如果一个指定名字的参数不是搜索内置的参数名, 搜索时会把该参数当作指定名字 tag 的属性来搜索, 如果包含一个名字为 id 的参数,Beautiful Soup 会搜索每个 tag 的” id” 属性.

搜索文档树

```
soup.find_all(id='link2')  
# [<a class="sister" href="http://example.com/lacie" i
```

搜索文档树

如果传入 href 参数,Beautiful Soup 会搜索每个 tag 的” href” 属性:

搜索文档树

```
soup.find_all(href=re.compile("elsie"))  
# [<a class="sister" href="http://example.com/elsie" i
```

搜索文档树

搜索指定名字的属性时可以使用的参数值包括字符串, 正则表达式, 列表, True .

搜索文档树

下面的例子在文档树中查找所有包含 id 属性的 tag, 无论 id 的值是什么:

搜索文档树

```
soup.find_all(id=True)
# [<a class="sister" href="http://example.com/elsie" i
#  <a class="sister" href="http://example.com/lacie" i
#  <a class="sister" href="http://example.com/tillie"
```

搜索文档树

使用多个指定名字的参数可以同时过滤 tag 的多个属性:

搜索文档树

```
soup.find_all(href=re.compile("elsie"), id='link1')  
# [<a class="sister" href="http://example.com/elsie" i
```


搜索文档树

有些 tag 属性在搜索不能使用, 比如 HTML5 中的 data-* 属性:

搜索文档树

```
data_soup = BeautifulSoup('<div data-foo="value">foo!<div data-foo="value">foo!')
data_soup.find_all(data-foo="value")
# SyntaxError: keyword can't be an expression
```

搜索文档树

但是可以通过 `find_all()` 方法的 `attrs` 参数定义一个字典参数来搜索包含特殊属性的 `tag`:

搜索文档树

```
data_soup.find_all(attrs={"data-foo": "value"})  
# [<div data-foo="value">foo!</div>]
```

搜索文档树

按照 CSS 类名搜索 tag 的功能非常实用, 但标识 CSS 类名的关键字 `class` 在 Python 中是保留字, 使用 `class` 做参数会导致语法错误. 从 Beautiful Soup 的 4.1.1 版本开始, 可以通过 `class_` 参数搜索有指定 CSS 类名的 tag:

搜索文档树

```
soup.find_all("a", class_="sister")  
# [<a class="sister" href="http://example.com/elsie" i  
#   <a class="sister" href="http://example.com/lacie" i  
#   <a class="sister" href="http://example.com/tillie"
```

搜索文档树

`class_` 参数同样接受不同类型的过滤器, 字符串, 正则表达式, 方法或 `True` :

搜索文档树

```
soup.find_all(class_=re.compile("itl"))  
# [<p class="title"><b>The Dormouse's story</b></p>]  
  
def has_six_characters(css_class):  
    return css_class is not None and len(css_class) == 6  
  
soup.find_all(class_=has_six_characters)  
# [<a class="sister" href="http://example.com/elsie" id="sister1">Elsie</a>,  
#  <a class="sister" href="http://example.com/lacie" id="sister2">Lacie</a>,  
#  <a class="sister" href="http://example.com/tillie" id="sister3">Tillie</a>]
```


搜索文档树

tag 的 class 属性是多值属性. 按照 CSS 类名搜索 tag 时, 可以分别搜索 tag 中的每个 CSS 类名:

搜索文档树

```
css_soup = BeautifulSoup('<p class="body strikethrough"></p>')
css_soup.find_all("p", class_="strikethrough")
# [<p class="body strikethrough"></p>]

css_soup.find_all("p", class_="body")
# [<p class="body strikethrough"></p>]
```

搜索文档树

搜索 class 属性时也可以通过 CSS 值完全匹配:

搜索文档树

```
css_soup.find_all("p", class_="body strikeout")  
# [<p class="body strikeout"></p>]
```

搜索文档树

完全匹配 class 的值时, 如果 CSS 类名的顺序与实际不符, 将搜索不到结果:

搜索文档树

```
soup.find_all("a", attrs={"class": "sister"})  
# [#  <a class="sister" href="http://example.com/lacie" i  
#  <a class="sister" href="http://example.com/tillie"
```

搜索文档树

通过 text 参数可以搜搜文档中的字符串内容. 与 name 参数的可选值一样, text 参数接受字符串, 正则表达式, 列表, True . 看例子:

搜索文档树

```
soup.find_all(text="Elsie")  
# [u'Elsie']
```

```
soup.find_all(text=["Tillie", "Elsie", "Lacie"])  
# [u'Elsie', u'Lacie', u'Tillie']
```

```
soup.find_all(text=re.compile("Dormouse"))  
[u"The Dormouse's story", u"The Dormouse's story"]
```

```
def is_the_only_string_within_a_tag(s):  
    """Return True if this string is the only child of  
    return (s == s.parent.string)
```

```
soup.find_all(text=is_the_only_string_within_a_tag)
```


搜索文档树

虽然 `text` 参数用于搜索字符串, 还可以与其它参数混合使用来过滤 `tag.Beautiful Soup` 会找到 `.string` 方法与 `text` 参数值相符的 `tag`. 下面代码用来搜索内容里面包含 “Elsie” 的 `<a>` 标签:

搜索文档树

```
soup.find_all("a", text="Elsie")  
# [<a href="http://example.com/elsie" class="sister" i
```

搜索文档树

`find_all()` 方法返回全部的搜索结构, 如果文档树很大那么搜索会很慢. 如果我们不需要全部结果, 可以使用 `limit` 参数限制返回结果的数量. 效果与 SQL 中的 `limit` 关键字类似, 当搜索到的结果数量达到 `limit` 的限制时, 就停止搜索返回结果.

搜索文档树

文档树中有 3 个 tag 符合搜索条件, 但结果只返回了 2 个, 因为我们限制了返回数量:

搜索文档树

```
soup.find_all("a", limit=2)
# [<a class="sister" href="http://example.com/elsie" i
#  <a class="sister" href="http://example.com/lacie" i
```

搜索文档树

调用 tag 的 `find_all()` 方法时,Beautiful Soup 会检索当前 tag 的所有子孙节点, 如果只想搜索 tag 的直接子节点, 可以使用参数 `recursive=False` .

搜索文档树

一段简单的文档:

搜索文档树

```
<html>
  <head>
    <title>
      The Dormouse's story
    </title>
  </head>
  ...
```


搜索文档树

是否使用 recursive 参数的搜索结果:

搜索文档树

```
soup.html.find_all("title")  
# [<title>The Dormouse's story</title>]  
  
soup.html.find_all("title", recursive=False)  
# []
```

搜索文档树

`find_all()` 几乎是 Beautiful Soup 中最常用的搜索方法, 所以我们定义了它的简写方法. BeautifulSoup 对象和 tag 对象可以被当作一个方法来使用, 这个方法的执行结果与调用这个对象的 `find_all()` 方法相同, 下面两行代码是等价的:

搜索文档树

```
soup.find_all("a")  
soup("a")
```

搜索文档树

这两行代码也是等价的:

搜索文档树

```
soup.title.find_all(text=True)  
soup.title(text=True)
```

搜索文档树

```
find( name , attrs , recursive , text , **kwargs )
```

搜索文档树

`find_all()` 方法将返回文档中符合条件的所有 tag, 尽管有时候我们只想得到一个结果. 比如文档中只有一个 `<body>` 标签, 那么使用 `find_all()` 方法来查找 `<body>` 标签就不太合适, 使用 `find_all` 方法并设置 `limit=1` 参数不如直接使用 `find()` 方法. 下面两行代码是等价的:

搜索文档树

```
soup.find_all('title', limit=1)
# [<title>The Dormouse's story</title>]

soup.find('title')
# <title>The Dormouse's story</title>
```

搜索文档树

唯一的区别是 `find_all()` 方法的返回结果是值包含一个元素的列表, 而 `find()` 方法直接返回结果.

搜索文档树

`find_all()` 方法没有找到目标是返回空列表, `find()` 方法找不到目标时, 返回 `None` .

搜索文档树

```
print(soup.find("nosuchtag"))  
# None
```

搜索文档树

`soup.head.title` 是 `tag` 的名字方法的简写. 这个简写的原理就是多次调用当前 `tag` 的 `find()` 方法:

搜索文档树

```
soup.head.title  
# <title>The Dormouse's story</title>  
  
soup.find("head").find("title")  
# <title>The Dormouse's story</title>
```

搜索文档树

```
find_parents( name , attrs , recursive , text , **kwargs )
```

搜索文档树

```
find_parent( name , attrs , recursive , text , **kwargs )
```


搜索文档树

我们已经用了很大篇幅来介绍 `find_all()` 和 `find()` 方法,Beautiful Soup 中还有 10 个用于搜索的 API. 它们中的五个用的是与 `find_all()` 相同的搜索参数, 另外 5 个与 `find()` 方法的搜索参数类似. 区别仅是它们搜索文档的不同部分.

搜索文档树

记住: `find_all()` 和 `find()` 只搜索当前节点的所有子节点, 孙子节点等.
`find_parents()` 和 `find_parent()` 用来搜索当前节点的父辈节点, 搜索方法与普通 tag 的搜索方法相同, 搜索文档搜索文档包含的内容. 我们从一个文档中的一个叶子节点开始:

搜索文档树

```
a_string = soup.find(text="Lacie")
a_string
# u'Lacie'

a_string.find_parents("a")
# [<a class="sister" href="http://example.com/lacie" i

a_string.find_parent("p")
# <p class="story">Once upon a time there were three l
# <a class="sister" href="http://example.com/elsie" i
# <a class="sister" href="http://example.com/lacie" i
# <a class="sister" href="http://example.com/tillie"
# and they lived at the bottom of a well.</p>
```



搜索文档树

文档中的一个 `<a>` 标签是是当前叶子节点的直接父节点, 所以可以被找到. 还有一个 `<p>` 标签, 是目标叶子节点的间接父辈节点, 所以也可以被找到. 包含 `class` 值为 "title" 的 `<p>` 标签不是不是目标叶子节点的父辈节点, 所以通过 `find_parents()` 方法搜索不到.

搜索文档树

`find_parent()` 和 `find_parents()` 方法会让人联想到 `.parent` 和 `.parents` 属性. 它们之间的联系非常紧密. 搜索父辈节点的方法实际上就是对 `.parents` 属性的迭代搜索.

搜索文档树

```
find_next_siblings( name , attrs , recursive , text , **kwargs )
```

搜索文档树

```
find_next_sibling( name , attrs , recursive , text , **kwargs )
```

搜索文档树

这 2 个方法通过 `.next_siblings` 属性对当 tag 的所有后面解析 [5] 的兄弟 tag 节点进行迭代, `find_next_siblings()` 方法返回所有符合条件的后面的兄弟节点, `find_next_sibling()` 只返回符合条件的后面的第一个 tag 节点.

搜索文档树

```
first_link = soup.a
first_link
# <a class="sister" href="http://example.com/elsie" id=

first_link.find_next_siblings("a")
# [<a class="sister" href="http://example.com/lacie" i
# <a class="sister" href="http://example.com/tillie"

first_story_paragraph = soup.find("p", "story")
first_story_paragraph.find_next_sibling("p")
# <p class="story">...</p>
```

搜索文档树

```
find_previous_siblings( name , attrs , recursive , text , **kwargs )
```

搜索文档树

```
find_previous_sibling( name , attrs , recursive , text , **kwargs )
```

搜索文档树

这 2 个方法通过 `.previous_siblings` 属性对当前 tag 的前面解析 [5] 的兄弟 tag 节点进行迭代, `find_previous_siblings()` 方法返回所有符合条件的前面的兄弟节点, `find_previous_sibling()` 方法返回第一个符合条件的前面的兄弟节点:

搜索文档树

```
last_link = soup.find("a", id="link3")
last_link
# <a class="sister" href="http://example.com/tillie" i

last_link.find_previous_siblings("a")
# [<a class="sister" href="http://example.com/lacie" i
# <a class="sister" href="http://example.com/elsie" i

first_story_paragraph = soup.find("p", "story")
first_story_paragraph.find_previous_sibling("p")
# <p class="title"><b>The Dormouse's story</b></p>
```

搜索文档树

```
find_all_next( name , attrs , recursive , text , **kwargs )
```

搜索文档树

```
find_next( name , attrs , recursive , text , **kwargs )
```

搜索文档树

这 2 个方法通过 `.next_elements` 属性对当前 tag 的之后的 [5] tag 和字符串进行迭代, `find_all_next()` 方法返回所有符合条件的节点, `find_next()` 方法返回第一个符合条件的节点:

搜索文档树

```
first_link = soup.a
first_link
# <a class="sister" href="http://example.com/elsie" id=

first_link.find_all_next(text=True)
# [u'Elsie', u',\n', u'Lacie', u' and\n', u'Tillie',
#  u';\nand they lived at the bottom of a well.', u'\n

first_link.find_next("p")
# <p class="story">...</p>
```

搜索文档树

第一个例子中, 字符串 “Elsie” 也被显示出来, 尽管它被包含在我们开始查找的 `<a>` 标签的里面. 第二个例子中, 最后一个 `<p>` 标签也被显示出来, 尽管它与我们开始查找位置的 `<a>` 标签不属于同一部分. 例子中, 搜索的重点是要匹配过滤器的条件, 并且在文档中出现的顺序而不是开始查找的元素的位置.

搜索文档树

```
find_all_previous( name , attrs , recursive , text , **kwargs )
```

搜索文档树

```
find_previous( name , attrs , recursive , text , **kwargs )
```

搜索文档树

这 2 个方法通过 `.previous_elements` 属性对当前节点前面 [5] 的 tag 和字符串进行迭代, `find_all_previous()` 方法返回所有符合条件的节点, `find_previous()` 方法返回第一个符合条件的节点.

搜索文档树

```
first_link = soup.a
first_link
# <a class="sister" href="http://example.com/elsie" id=

first_link.find_all_previous("p")
# [<p class="story">Once upon a time there were three
#  <p class="title"><b>The Dormouse's story</b></p>]

first_link.find_previous("title")
# <title>The Dormouse's story</title>
```

搜索文档树

`find_all_previous("p")` 返回了文档中的第一段 (`class="title"` 的那段), 但还返回了第二段, `<p>` 标签包含了我们开始查找的 `<a>` 标签. 不要惊讶, 这段代码的功能是查找所有出现在指定 `<a>` 标签之前的 `<p>` 标签, 因为这个 `<p>` 标签包含了开始的 `<a>` 标签, 所以 `<p>` 标签一定是在 `<a>` 之前出现的.

搜索文档树

Beautiful Soup 支持大部分的 CSS 选择器 [6], 在 Tag 或 BeautifulSoup 对象的.select() 方法中传入字符串参数, 即可使用 CSS 选择器的语法找到 tag:

搜索文档树

```
soup.select("title")  
# [<title>The Dormouse's story</title>]  
  
soup.select("p nth-of-type(3)")  
# [<p class="story">...</p>]
```

搜索文档树

通过 tag 标签逐层查找:

搜索文档树

```
soup.select("body a")  
# [<a class="sister" href="http://example.com/elsie" i  
#   <a class="sister" href="http://example.com/lacie"  
#   <a class="sister" href="http://example.com/tillie"  
  
soup.select("html head title")  
# [<title>The Dormouse's story</title>]
```

搜索文档树

找到某个 tag 标签下的直接子标签 [6] :

搜索文档树

```
soup.select("head > title")
# [<title>The Dormouse's story</title>]

soup.select("p > a")
# [<a class="sister" href="http://example.com/elsie" i
#  <a class="sister" href="http://example.com/lacie"
#  <a class="sister" href="http://example.com/tillie"

soup.select("p > a:nth-of-type(2)")
# [<a class="sister" href="http://example.com/lacie" i

soup.select("p > #link1")
# [<a class="sister" href="http://example.com/elsie" i
```

搜索文档树

找到兄弟节点标签:

搜索文档树

```
soup.select("#link1 ~ .sister")
# [<a class="sister" href="http://example.com/lacie" i
#  <a class="sister" href="http://example.com/tillie" i

soup.select("#link1 + .sister")
# [<a class="sister" href="http://example.com/lacie" i
```

搜索文档树

通过 CSS 的类名查找:

搜索文档树

```
soup.select(".sister")  
# [#   #     


```
soup.select\("\[class~=sister\]"\)
\[# #
```


```

搜索文档树

通过 tag 的 id 查找:

搜索文档树

```
soup.select("#link1")  
# [  
soup.select("a#link2")  
# [
```

搜索文档树

通过是否存在某个属性来查找:

搜索文档树

```
soup.select('a[href]')  
# [<a class="sister" href="http://example.com/elsie" i  
#   <a class="sister" href="http://example.com/lacie" i  
#   <a class="sister" href="http://example.com/tillie"
```

搜索文档树

通过属性的值来查找:

搜索文档树

```
soup.select('a[href="http://example.com/elsie"]')
# [<a class="sister" href="http://example.com/elsie" i

soup.select('a[href^="http://example.com/"]')
# [<a class="sister" href="http://example.com/elsie" i
# <a class="sister" href="http://example.com/lacie" i
# <a class="sister" href="http://example.com/tillie"

soup.select('a[href$="tillie"]')
# [<a class="sister" href="http://example.com/tillie"

soup.select('a[href*=".com/el"]')
# [<a class="sister" href="http://example.com/elsie" i
```

搜索文档树

通过语言设置来查找:

搜索文档树

```
multilingual_markup = """
    <p lang="en">Hello</p>
    <p lang="en-us">Howdy, y'all</p>
    <p lang="en-gb">Pip-pip, old fruit</p>
    <p lang="fr">Bonjour mes amis</p>
    """

multilingual_soup = BeautifulSoup(multilingual_markup)
multilingual_soup.select('p[lang|=en]')
# [<p lang="en">Hello</p>,
#  <p lang="en-us">Howdy, y'all</p>,
#  <p lang="en-gb">Pip-pip, old fruit</p>]
```

搜索文档树

对于熟悉 CSS 选择器语法的人来说这是个非常方便的方法. Beautiful Soup 也支持 CSS 选择器 API, 如果你仅仅需要 CSS 选择器的功能, 那么直接使用 lxml 也可以, 而且速度更快, 支持更多的 CSS 选择器语法, 但 Beautiful Soup 整合了 CSS 选择器的语法和自身方便使用 API.

Outline

- 1 快速开始
- 2 如何使用
- 3 对象的种类
- 4 遍历文档树
- 5 搜索文档树
- 6 修改文档树**
- 7 输出
- 8 指定文档解析器
- 9 编码
- 10 解析部分文档

修改文档树

Beautiful Soup 的强项是文档树的搜索, 但同时也可以方便的修改文档树

修改文档树

在 Attributes 的章节中已经介绍过这个功能, 但是再看一遍也无妨. 重命名一个 tag, 改变属性的值, 添加或删除属性:

修改文档树

```
soup = BeautifulSoup('<b class="boldest">Extremely bold')
tag = soup.b

tag.name = "blockquote"
tag['class'] = 'verybold'
tag['id'] = 1
tag
# <blockquote class="verybold" id="1">Extremely bold</blockquote>

del tag['class']
del tag['id']
tag
# <blockquote>Extremely bold</blockquote>
```

修改文档树

给 tag 的.string 属性赋值, 就相当于用当前的内容替代了原来的内容:

修改文档树

```
markup = '<a href="http://example.com/">I linked to <i  
soup = BeautifulSoup(markup)
```

```
tag = soup.a  
tag.string = "New link text."  
tag
```

```
# <a href="http://example.com/">New link text.</a>
```


修改文档树

注意: 如果当前的 tag 包含了其它 tag, 那么给它的.string 属性赋值会覆盖掉原有的所有内容包括子 tag

修改文档树

Tag.append() 方法向 tag 中添加内容, 就好像 Python 的列表的.append() 方法:

修改文档树

```
soup = BeautifulSoup("<a>Foo</a>")  
soup.a.append("Bar")
```

```
soup
```

```
# <html><head></head><body><a>FooBar</a></body></html>
```

```
soup.a.contents
```

```
# [u'Foo', u'Bar']
```

修改文档树

如果想添加一段文本内容到文档中也没问题, 可以调用 Python 的 `append()` 方法或调用工厂方法 `BeautifulSoup.new_string()` :

修改文档树

```
soup = BeautifulSoup("<b></b>")
tag = soup.b
tag.append("Hello")
new_string = soup.new_string(" there")
tag.append(new_string)
tag
# <b>Hello there.</b>
tag.contents
# [u'Hello', u' there']
```

修改文档树

如果想要创建一段注释, 或 NavigableString 的任何子类, 将子类作为 new_string() 方法的第二个参数传入:

修改文档树

```
from bs4 import Comment
new_comment = soup.new_string("Nice to see you.", Comm
tag.append(new_comment)
tag
# <b>Hello there<!--Nice to see you.--></b>
tag.contents
# [u'Hello', u' there', u'Nice to see you.']
```

修改文档树

这是 Beautiful Soup 4.2.1 中新增的方法

修改文档树

创建一个 tag 最好的方法是调用工厂方法 BeautifulSoup.new_tag() :

修改文档树

```
soup = BeautifulSoup("<b></b>")
original_tag = soup.b

new_tag = soup.new_tag("a", href="http://www.example.com")
original_tag.append(new_tag)
original_tag
# <b><a href="http://www.example.com"></a></b>

new_tag.string = "Link text."
original_tag
# <b><a href="http://www.example.com">Link text.</a></b>
```

修改文档树

第一个参数作为 tag 的 name, 是必填, 其它参数选填

修改文档树

`Tag.insert()` 方法与 `Tag.append()` 方法类似, 区别是不会把新元素添加到父节点.`contents` 属性的最后, 而是把元素插入到指定的位置. 与 Python 列表总的.`insert()` 方法的用法下同:

修改文档树

```
markup = '<a href="http://example.com/">I linked to <i>  
soup = BeautifulSoup(markup)  
tag = soup.a  
  
tag.insert(1, "but did not endorse ")  
tag  
# <a href="http://example.com/">I linked to but did not  
tag.contents  
# [u'I linked to ', u'but did not endorse', <i>example
```

修改文档树

`insert_before()` 方法在当前 tag 或文本节点前插入内容:

修改文档树

```
soup = BeautifulSoup("<b>stop</b>")
tag = soup.new_tag("i")
tag.string = "Don't"
soup.b.string.insert_before(tag)
soup.b
# <b><i>Don't</i>stop</b>
```

修改文档树

`insert_after()` 方法在当前 tag 或文本节点后插入内容:

修改文档树

```
soup.b.i.insert_after(soup.new_string(" ever "))
soup.b
# <b><i>Don't</i> ever stop</b>
soup.b.contents
# [<i>Don't</i>, u' ever ', u'stop']
```

修改文档树

Tag.clear() 方法移除当前 tag 的内容:

修改文档树

```
markup = '<a href="http://example.com/">I linked to <i
soup = BeautifulSoup(markup)
tag = soup.a

tag.clear()
tag
# <a href="http://example.com/"></a>
```

修改文档树

`PageElement.extract()` 方法将当前 tag 移除文档树, 并作为方法结果返回:

修改文档树

```
markup = '<a href="http://example.com/">I linked to <i>example.com</i></a>'
soup = BeautifulSoup(markup)
a_tag = soup.a

i_tag = soup.i.extract()

a_tag
# <a href="http://example.com/">I linked to</a>

i_tag
# <i>example.com</i>

print(i_tag.parent)
None
```

修改文档树

这个方法实际上产生了 2 个文档树: 一个是用来解析原始文档的 BeautifulSoup 对象, 另一个是被移除并且返回的 tag. 被移除并返回的 tag 可以继续调用 extract 方法:

修改文档树

```
my_string = i_tag.string.extract()  
my_string  
# u'example.com'  
  
print(my_string.parent)  
# None  
i_tag  
# <i></i>
```

修改文档树

Tag.decompose() 方法将当前节点移除文档树并完全销毁:

修改文档树

```
markup = '<a href="http://example.com/">I linked to <i  
soup = BeautifulSoup(markup)  
a_tag = soup.a  
  
soup.i.decompose()  
  
a_tag  
# <a href="http://example.com/">I linked to</a>
```

修改文档树

`PageElement.replace_with()` 方法移除文档树中的某段内容, 并用新 tag 或文本节点替代它:

修改文档树

```
markup = '<a href="http://example.com/">I linked to <i  
soup = BeautifulSoup(markup)  
a_tag = soup.a  
  
new_tag = soup.new_tag("b")  
new_tag.string = "example.net"  
a_tag.i.replace_with(new_tag)  
  
a_tag  
# <a href="http://example.com/">I linked to <b>example
```

修改文档树

`replace_with()` 方法返回被替代的 tag 或文本节点, 可以用来浏览或添加到文档树其它地方

修改文档树

PageElement.wrap() 方法可以对指定的 tag 元素进行包装 [8]，并返回包装后的结果：

修改文档树

```
soup = BeautifulSoup("<p>I wish I was bold.</p>")
soup.p.string.wrap(soup.new_tag("b"))
# <b>I wish I was bold.</b>

soup.p.wrap(soup.new_tag("div"))
# <div><p><b>I wish I was bold.</b></p></div>
```

修改文档树

该方法在 Beautiful Soup 4.0.5 中添加

修改文档树

Tag.unwrap() 方法与 wrap() 方法相反. 将移除 tag 内的所有 tag 标签, 该方法常被用来进行标记的解包:

修改文档树

```
markup = '<a href="http://example.com/">I linked to <i  
soup = BeautifulSoup(markup)  
a_tag = soup.a  
  
a_tag.i.unwrap()  
a_tag  
# <a href="http://example.com/">I linked to example.co
```

修改文档树

与 `replace_with()` 方法相同, `unwrap()` 方法返回被移除的 tag

Outline

- 1 快速开始
- 2 如何使用
- 3 对象的种类
- 4 遍历文档树
- 5 搜索文档树
- 6 修改文档树
- 7 输出**
- 8 指定文档解析器
- 9 编码
- 10 解析部分文档

输出

`prettify()` 方法将 BeautifulSoup 的文档树格式化后以 Unicode 编码输出, 每个 XML/HTML 标签都独占一行

输出

```
markup = '<a href="http://example.com/">I linked to <i>  
soup = BeautifulSoup(markup)  
soup.prettify()  
# '<html>\n <head>\n </head>\n <body>\n  <a href="http  
  
print(soup.prettify())  
# <html>  
#   <head>  
#   </head>  
#   <body>  
#     <a href="http://example.com/">  
#       I linked to  
#       <i>  
#         example.com
```

输出

BeautifulSoup 对象和它的 tag 节点都可以调用 `prettify()` 方法:

输出

```
print(soup.a.prettify())  
# <a href="http://example.com/">  
#   I linked to  
#   <i>  
#     example.com  
#   </i>  
# </a>
```

输出

如果只想得到结果字符串, 不重视格式, 那么可以对一个 BeautifulSoup 对象或 Tag 对象使用 Python 的 `unicode()` 或 `str()` 方法:

输出

```
str(soup)
# '<html><head></head><body><a href="http://example.co

unicode(soup.a)
# u'<a href="http://example.com/">I linked to <i>examp
```

输出

`str()` 方法返回 UTF-8 编码的字符串, 可以指定编码的设置.

输出

还可以调用 `encode()` 方法获得字节码或调用 `decode()` 方法获得 Unicode.

输出

Beautiful Soup 输出是会将 HTML 中的特殊字符转换成 Unicode, 比如 “&lquot;” :

输出

```
soup = BeautifulSoup("&ldquo;Dammit!&rdquo; he said.")
unicode(soup)
# u'<html><head></head><body>\u201cDammit!\u201d he sa
```

输出

如果将文档转换成字符串,Unicode 编码会被编码成 UTF-8. 这样就无法正确显示 HTML 特殊字符了:

输出

```
str(soup)
```

```
# ' <html><head></head><body>\xe2\x80\x9cDammit!\xe2\x8
```

输出

如果只想得到 tag 中包含的文本内容, 那么可以调用 `get_text()` 方法, 这个方法获取到 tag 中包含的所有文本内容包括子孙 tag 中的内容, 并将结果作为 Unicode 字符串返回:

输出

```
markup = '<a href="http://example.com/">\nI linked to  
soup = BeautifulSoup(markup)
```

```
soup.get_text()  
u'\nI linked to example.com\n'  
soup.i.get_text()  
u'example.com'
```

输出

可以通过参数指定 tag 的文本内容的分隔符:

输出

```
# soup.get_text("|")  
u'\nI linked to |example.com|\n'
```

输出

还可以去除获得文本内容的前后空白:

输出

```
# soup.get_text("|", strip=True)  
u'I linked to|example.com'
```

输出

或者使用`.stripped_strings` 生成器, 获得文本列表后手动处理列表:

输出

```
[text for text in soup.stripped_strings]  
# [u'I linked to', u'example.com']
```

Outline

- 1 快速开始
- 2 如何使用
- 3 对象的种类
- 4 遍历文档树
- 5 搜索文档树
- 6 修改文档树
- 7 输出
- 8 指定文档解析器**
- 9 编码
- 10 解析部分文档

指定文档解析器

如果仅是想要解析 HTML 文档, 只要用文档创建 BeautifulSoup 对象就可以了. BeautifulSoup 会自动选择一个解析器来解析文档. 但是还可以通过参数指定使用那种解析器来解析当前文档.

指定文档解析器

BeautifulSoup 第一个参数应该是要被解析的文档字符串或是文件句柄, 第二个参数用来标识怎样解析文档. 如果第二个参数为空, 那么 BeautifulSoup 根据当前系统安装的库自动选择解析器, 解析器的优先数序: lxml, html5lib, Python 标准库. 在下面两种条件下解析器优先顺序会变化:

指定文档解析器

安装解析器章节介绍了可以使用哪种解析器, 以及如何安装.

指定文档解析器

如果指定的解析器没有安装,Beautiful Soup 会自动选择其它方案. 目前只有 lxml 解析器支持 XML 文档的解析, 在没有安装 lxml 库的情况下, 创建 BeautifulSoup 对象时无论是否指定使用 lxml, 都无法得到解析后的对象

指定文档解析器

Beautiful Soup 为不同的解析器提供了相同的接口, 但解析器本身时有区别的. 同一篇文档被不同的解析器解析后可能会生成不同结构的树型文档. 区别最大的是 HTML 解析器和 XML 解析器, 看下面片段被解析成 HTML 结构:

指定文档解析器

```
BeautifulSoup("<a><b /></a>")  
# <html><head></head><body><a><b></b></a></body></html>
```

指定文档解析器

因为空标签 `<b />` 不符合 HTML 标准, 所以解析器把它解析成 `<b></b>`

指定文档解析器

同样的文档使用 XML 解析如下 (解析 XML 需要安装 lxml 库). 注意, 空标签 `<b />` 依然被保留, 并且文档前添加了 XML 头, 而不是被包含在 `<html>` 标签内:

指定文档解析器

```
BeautifulSoup("<a><b /></a>", "xml")  
# <?xml version="1.0" encoding="utf-8"?>  
# <a><b/></a>
```

指定文档解析器

HTML 解析器之间也有区别, 如果被解析的 HTML 文档是标准格式, 那么解析器之间没有任何差别, 只是解析速度不同, 结果都会返回正确的文档树.

指定文档解析器

但是如果被解析文档不是标准格式, 那么不同的解析器返回结果可能不同. 下面例子中, 使用 lxml 解析错误格式的文档, 结果 `</p>` 标签被直接忽略掉了:

指定文档解析器

```
BeautifulSoup("<a></p>", "lxml")  
# <html><body><a></a></body></html>
```

指定文档解析器

使用 `html5lib` 库解析相同文档会得到不同的结果:

指定文档解析器

```
BeautifulSoup("<a></p>", "html5lib")  
# <html><head></head><body><a><p></p></a></body></html>
```

指定文档解析器

html5lib 库没有忽略掉 `</p>` 标签, 而是自动补全了标签, 还给文档树添加了 `<head>` 标签.

指定文档解析器

使用 pyhton 内置库解析结果如下:

指定文档解析器

```
BeautifulSoup("<a></p>", "html.parser")  
# <a></a>
```

指定文档解析器

与 lxml [7] 库类似的,Python 内置库忽略掉了 `</p>` 标签,与 `html5lib` 库不同的是标准库没有尝试创建符合标准的文档格式或将文档片段包含在 `<body>` 标签内,与 `lxml` 不同的是标准库甚至连 `<html>` 标签都没有尝试去添加.

指定文档解析器

因为文档片段“<a></p>”是错误格式,所以以上解析方式都能算作”正确”,html5lib 库使用的是 HTML5 的部分标准,所以最接近”正确”.不过所有解析器的结构都能够被认为是”正常”的.

指定文档解析器

不同的解析器可能影响代码执行结果, 如果在分发给别人的代码中使用了 BeautifulSoup , 那么最好注明使用了哪种解析器, 以减少不必要的麻烦.

Outline

- 1 快速开始
- 2 如何使用
- 3 对象的种类
- 4 遍历文档树
- 5 搜索文档树
- 6 修改文档树
- 7 输出
- 8 指定文档解析器
- 9 编码**
- 10 解析部分文档

编码

任何 HTML 或 XML 文档都有自己的编码方式, 比如 ASCII 或 UTF-8, 但是使用 BeautifulSoup 解析后, 文档都被转换成了 Unicode:

编码

```
markup = "<h1>Sacr\xc3\xa9 bleu!</h1>"
soup = BeautifulSoup(markup)
soup.h1
# <h1>Sacr  bleu!</h1>
soup.h1.string
# u'Sacr  bleu!'
```

编码

这不是魔术 (但很神奇),Beautiful Soup 用了编码自动检测子库来识别当前文档编码并转换成 Unicode 编码. BeautifulSoup 对象的 `original_encoding` 属性记录了自动识别编码的结果:

编码

```
soup.original_encoding  
'utf-8'
```

编码

编码自动检测功能大部分时候都能猜对编码格式, 但有时候也会出错. 有时候即使猜测正确, 也是在逐个字节的遍历整个文档后才猜对的, 这样很慢. 如果预先知道文档编码, 可以设置编码参数来减少自动检查编码出错的概率并且提高文档解析速度. 在创建 BeautifulSoup 对象的时候设置 `from_encoding` 参数.

编码

下面一段文档用了 ISO-8859-8 编码方式, 这段文档太短, 结果 Beautiful Soup 以为文档是用 ISO-8859-7 编码:

编码

```
markup = b"<h1>\xed\xe5\xec\xf9</h1>"
soup = BeautifulSoup(markup)
soup.h1
<h1></h1>
soup.original_encoding
'ISO-8859-7'
```

编码

通过传入 `from_encoding` 参数来指定编码方式:

编码

```
soup = BeautifulSoup(markup, from_encoding="iso-8859-8")
soup.h1
<h1></h1>
soup.original_encoding
'iso8859-8'
```

编码

少数情况下 (通常是 UTF-8 编码的文档中包含了其它编码格式的文件), 想获得正确的 Unicode 编码就不得不将文档中少数特殊编码字符替换成特殊 Unicode 编码, “REPLACEMENT CHARACTER” (U+FFFD,) [9] . 如果 Beautiful Soup 猜测文档编码时作了特殊字符的替换, 那么 BeautifulSoup 会把 UnicodeDammit 或 BeautifulSoup 对象的 `contains_replacement_characters` 属性标记为 `True` . 这样就可以知道当前文档进行 Unicode 编码后丢失了一部分特殊内容字符. 如果文档中包含而 `contains_replacement_characters` 属性是 `False` , 则表示就是文档中原来的字符, 不是转码失败.

编码

通过 Beautiful Soup 输出文档时, 不管输入文档是什么编码方式, 输出编码均为 UTF-8 编码, 下面例子输入文档是 Latin-1 编码:

编码

```
markup = b'''
<html>
  <head>
    <meta content="text/html; charset=ISO-Latin-1" htt
  </head>
  <body>
    <p>Sacr\xe9 bleu!</p>
  </body>
</html>
'''

soup = BeautifulSoup(markup)
print(soup.prettify())
# <html>
```

编码

注意, 输出文档中的 `<meta>` 标签的编码设置已经修改成了与输出编码一致的 UTF-8.

编码

如果不想用 UTF-8 编码输出, 可以将编码方式传入 `prettify()` 方法:

编码

```
print(soup.prettify("latin-1"))  
# <html>  
#   <head>  
#     <meta content="text/html; charset=latin-1" http-equiv="content-type">  
#   ...
```

编码

还可以调用 BeautifulSoup 对象或任意节点的 `encode()` 方法, 就像 Python 的字符串调用 `encode()` 方法一样:

编码

```
soup.p.encode("latin-1")  
# '<p>Sacr\xe9 bleu!</p>'  
  
soup.p.encode("utf-8")  
# '<p>Sacr\xc3\xa9 bleu!</p>'
```

编码

如果文档中包含当前编码不支持的字符, 那么这些字符将呗转换成一系列 XML 特殊字符引用, 下面例子中包含了 Unicode 编码字符 SNOWMAN:

编码

```
markup = u"<b>\N{SNOWMAN}</b>"  
snowman_soup = BeautifulSoup(markup)  
tag = snowman_soup.b
```


编码

SNOWMAN 字符在 UTF-8 编码中可以正常显示 (看上去像是), 但有些编码不支持 SNOWMAN 字符, 比如 ISO-Latin-1 或 ASCII, 那么在这些编码中 SNOWMAN 字符会被转换成 “☃” :

编码

```
print(tag.encode("utf-8"))  
# <b></b>  
  
print tag.encode("latin-1")  
# <b>&#9731;</b>  
  
print tag.encode("ascii")  
# <b>&#9731;</b>
```

编码

编码自动检测功能可以在 BeautifulSoup 以外使用, 检测某段未知编码时, 可以使用这个方法:

编码

```
from bs4 import UnicodeDammit
dammit = UnicodeDammit("Sacr\xc3\xa9 bleu!")
print(dammit.unicode_markup)
# Sacré bleu!
dammit.original_encoding
# 'utf-8'
```

编码

如果 Python 中安装了 `chardet` 或 `cchardet` 那么编码检测功能的准确率将大大提高. 输入的字符越多, 检测结果越精确, 如果事先猜测到一些可能编码, 那么可以将猜测的编码作为参数, 这样将优先检测这些编码:

编码

```
dammit = UnicodeDammit("Sacré bleu!", ["latin-1", "  
print(dammit.unicode_markup)  
# Sacré bleu!  
dammit.original_encoding  
# 'latin-1'
```

编码

编码自动检测功能中有 2 项功能是 BeautifulSoup 库中用不到的

编码

使用 Unicode 时,Beautiful Soup 还会智能的把引号 [10] 转换成 HTML 或 XML 中的特殊字符:

编码

```
markup = b"<p>I just \x93love\x94 Microsoft Word\x92s
UnicodeDammit(markup, ["windows-1252"], smart_quotes_t
# u'<p>I just &ldquo;love&rdquo; Microsoft Word&rsquo;
UnicodeDammit(markup, ["windows-1252"], smart_quotes_t
# u'<p>I just &#x201C;love&#x201D; Microsoft Word&#x20
```

编码

也可以把引号转换为 ASCII 码:

编码

```
UnicodeDammit(markup, ["windows-1252"], smart_quotes_t  
# u'<p>I just "love" Microsoft Word\'s smart quotes</p>
```

编码

很有用的功能, 但是 Beautiful Soup 没有使用这种方式. 默认情况下,Beautiful Soup 把引号转换成 Unicode:

编码

```
UnicodeDammit(markup, ["windows-1252"]).unicode_markup  
# u'<p>I just \u201clove\u201d Microsoft Word\u2019s s
```

编码

有时文档的大部分都是用 UTF-8, 但同时还包含了 Windows-1252 编码的字符, 就像微软的智能引号 [10] 一样. 一些包含多个信息的来源网站容易出现这种情况. `UnicodeDammit.detwingle()` 方法可以把这类文档转换成纯 UTF-8 编码格式, 看个简单的例子:

编码

```
snowmen = (u"\N{SNOWMAN}" * 3)
quote = (u"\N{LEFT DOUBLE QUOTATION MARK}I like snowme
doc = snowmen.encode("utf8") + quote.encode("windows_1
```

编码

这段文档很杂乱,snowmen 是 UTF-8 编码,引号是 Windows-1252 编码,直接输出时不能同时显示 snowmen 和引号,因为它们编码不同:

编码

```
print(doc)
# I like snowmen!

print(doc.decode("windows-1252"))
# âââ "I like snowmen!"
```

编码

如果对这段文档用 UTF-8 解码就会得到 UnicodeDecodeError 异常, 如果用 Windows-1252 解码就回得到一堆乱码. 幸好, UnicodeDammit.detwingle() 方法会把这段字符串转换成 UTF-8 编码, 允许我们同时显示出文档中的 snowmen 和引号:

编码

```
new_doc = UnicodeDammit.detwingle(doc)
print(new_doc.decode("utf8"))
# “I like snowmen!”
```

编码

UnicodeDammit.detwingle() 方法只能解码包含在 UTF-8 编码中的 Windows-1252 编码内容, 但这解决了最常见的一类问题.

编码

在创建 BeautifulSoup 或 UnicodeDammit 对象前一定要先对文档调用 `UnicodeDammit.detwingle()` 确保文档的编码方式正确. 如果尝试去解析一段包含 Windows-1252 编码的 UTF-8 文档, 就会得到一堆乱码, 比如: `âââ “I like snowmen!”` .

编码

UnicodeDammit.detwingle() 方法在 BeautifulSoup 4.1.0 版本中新增

Outline

- 1 快速开始
- 2 如何使用
- 3 对象的种类
- 4 遍历文档树
- 5 搜索文档树
- 6 修改文档树
- 7 输出
- 8 指定文档解析器
- 9 编码
- 10 解析部分文档**

解析部分文档

如果仅仅因为想要查找文档中的 `<a>` 标签而将整片文档进行解析, 实在是浪费内存和时间. 最快的方法是从一开始就把 `<a>` 标签以外的东西都忽略掉. `SoupStrainer` 类可以定义文档的某段内容, 这样搜索文档时就不必先解析整篇文档, 只会解析在 `SoupStrainer` 中定义过的文档. 创建一个 `SoupStrainer` 对象并作为 `parse_only` 参数给 `BeautifulSoup` 的构造方法即可.

解析部分文档

SoupStrainer 类接受与典型搜索方法相同的参数：name , attrs , recursive , text , **kwargs 。下面举例说明三种 SoupStrainer 对象：

解析部分文档

```
from bs4 import SoupStrainer

only_a_tags = SoupStrainer("a")

only_tags_with_id_link2 = SoupStrainer(id="link2")

def is_short_string(string):
    return len(string) < 10

only_short_strings = SoupStrainer(text=is_short_string)
```

解析部分文档

再拿“爱丽丝”文档来举例，来看看使用三种 SoupStrainer 对象做参数会有什么不同：

解析部分文档

```
html_doc = """
<html><head><title>The Dormouse's story</title></head>

<p class="title"><b>The Dormouse's story</b></p>

<p class="story">Once upon a time there were three lit
<a href="http://example.com/elsie" class="sister" id="
<a href="http://example.com/lacie" class="sister" id="
<a href="http://example.com/tillie" class="sister" id=
and they lived at the bottom of a well.</p>

<p class="story">...</p>
"""
```

解析部分文档

还可以将 SoupStrainer 作为参数传入搜索文档树中提到的方法. 这可能不是个常用用法, 所以还是提一下:

解析部分文档

```
soup = BeautifulSoup(html_doc)
soup.find_all(only_short_strings)
# [u'\n\n', u'\n\n', u'Elsie', u',\n', u'Lacie', u' an
# u'\n\n', u'...', u'\n']
```